ShieldMate — Development Plan

Oct 29, 2026 · @Radhika

Build a basic working product by hour 8 of the 24-hour build (Saturday to Sunday), then add one tested increment at a time; every increment leaves a demo that runs. Assumes no code is written before H0 and the demo follows H24.

Ground rules

These rules keep the product runnable at every point, so a late increment never costs you the demo.

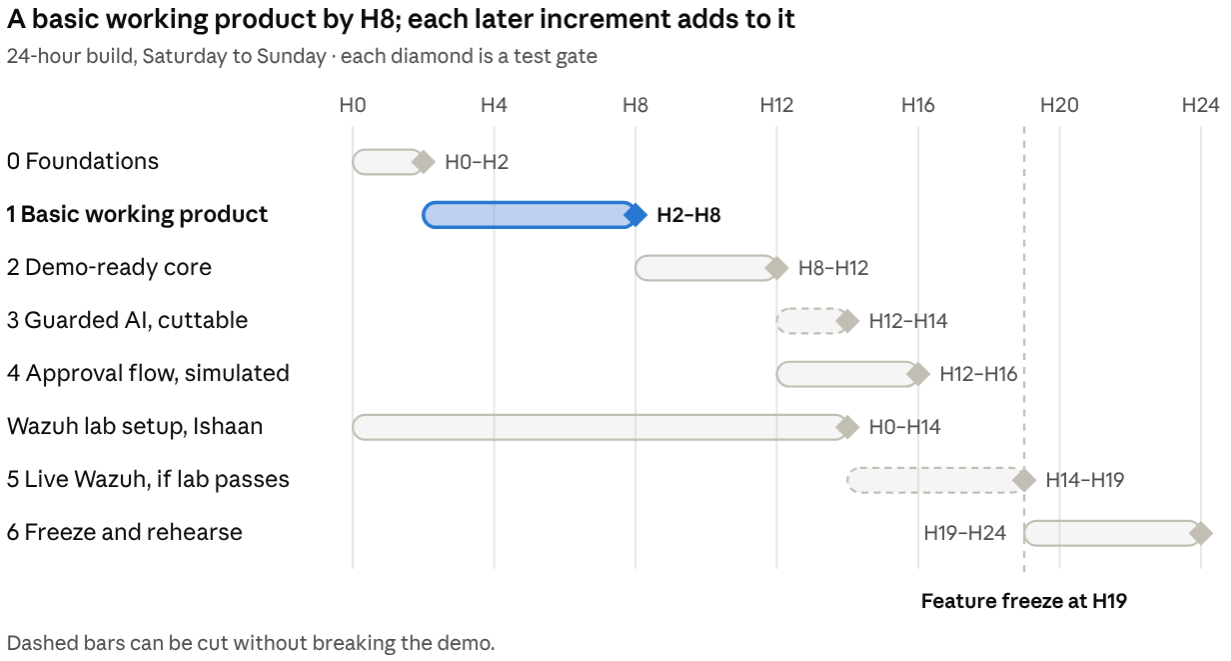
1. **Main always runs the demo.** Work on branches; merge only when that increment's tests in the Testing Plan pass.
2. **Contract first.** The API contract in the Software Design is frozen in Increment 0. Changing it needs all four of you to agree, and the contract tests are updated in the same merge.
3. **New features ship behind a flag.** `AI_REWORD`, `ACTIONS`, `ALERT_SOURCE=wazuh` default off, so unfinished work can merge without changing what the demo does.
4. **Integrate at every gate.** At H2, H8, H12, H16 and H19 everyone merges and you run the full demo story together on the demo laptop and two real phones.
5. **An increment is done when its gate passes**, not when the code is written. Gate test IDs (T0–T6) match increment numbers (0–6).
6. **Long-lead work starts early.** Ishaan sets up the Wazuh lab from H0, even though it is only connected in Increment 5. It is the longest and least predictable task.

Rest in shifts. Each person takes one 90-minute rest between H12 and H19, staggered so no two people rest at once, and Ishaan never rests during the lab gate.

Hours	Increment	Gate at the end
H0–H2	0. Foundations	T0
H2–H8	1. **Basic working product**	T1
H8–H12	2. Demo-ready core	T2
H12–H14	3. Guarded AI rewording (first to cut)	T3
H12–H16	4. Approval flow, simulated	T4
H14–H19	5. Live Wazuh, only if the lab gate passes at H14	T5

Hours	Increment	Gate at the end
H19	**Feature freeze**	
H19–H24	6. Fix, rehearse, demo	T6

Plan at a glance



24-hour build plan · 7 increments, lab setup in parallel

Increments 3 and 4 run in parallel at H12, and the lab setup runs alongside everything until its gate at H14.

Increment 0: Foundations

Goal: all four of you can build in parallel against one frozen contract. **When:** H0–H2. **Gate:** T0.

Owner	Task	Output
Kishaann	FastAPI skeleton; SQLite schema with **every** table, including live-mode ones; a stub for every contract route returning fixture JSON; the shared error shape	`backend/main.py`, `db.py`, `fixtures/`

Owner	Task	Output
Kishaann	Seed loader from `seed/clinic.json`; `POST /demo/reset`	Reset works on an empty database
Abhiram	Vite + React app; the 5 routes as placeholder pages; `api.js` mirroring the contract; `usePoll` hook; production build served by FastAPI	`frontend/`; `./start.sh`
Cetas	Ransomware scenario file (design doc, Appendix A) and its data models	`scenarios/ransomware_clinic.json`, `drills/models.py`
Cetas	Synthetic backup generator: 50 originals, manifest of hashes, backup with 2 altered files, metadata 9 days old, always connected	`demo_backup/make.py`
Ishaan	Replay file (design doc, Appendix B) and its data model	`replay/alerts.json`
Ishaan	Start the Wazuh lab: manager + Windows VM with agent on his laptop	Agent shows Active in the Wazuh dashboard
All	Repo, branch rules, `make test` running pytest	One command runs every test

Done when: `./start.sh` on the demo laptop serves the start page to a phone on the hotspot, and every API route returns fixture data that the frontend can render.

Increment 1: Basic working product

Goal: the whole demo story runs end to end in replay mode, on two real phones and the laptop. Plain screens are fine; every step must work for real. **When:** H2–H8. **Gate:** T1.

This is the product you could demo if nothing else got built. Everything after it adds to it without replacing it.

Owner	Task	Output
Ishaan	Replay ingest: Backstage button injects the next click's records	`POST /demo/replay/{type}` is real
Kishaann	Normalise alerts; look up the lesson by rule ID; fill `{agent_name}`, `{count}` and similar into the text; cards per role	`GET /alerts?role=` is real
Kishaann	Done creates an owner notification	`POST /alerts/{id}/done`, `GET /notifications` are real
Cetas	Drill engine: create with a 4-digit code, join, state machine (lobby, open, review, finished), answer once, feedback, owner-only advance, score by phase	All `/drills` routes are real
Cetas	Restore checker: sandbox copy, hash compare, age and exposure checks, saved result; the step-3 trigger	`/restore/run`, `/restore/latest` are real
Abhiram	Start screen, staff alert card (steps, Call IT, Done), owner dashboard (cards, notifications, drill panel, restore result), drill screens for staff and owner, Backstage buttons	All 5 routes working, unstyled is fine

Not in this increment: noise filtering, grouping, the From-the-log box, languages, impact meter, AI, any live-mode work.

Done when: two people with phones and one at the laptop complete the full demo story twice in a row, with Reset in between, and nothing needs a restart.

Increment 2: Demo-ready core

Goal: the basic product looks and behaves like the wireframes, and survives the things that go wrong on stage. **When:** H8–H12. **Gate:** T2.

Owner	Task
Kishaann	Noise filter (level below 7, or no lesson) with a hidden-events count; grouping of repeats within 10 minutes into one card

Owner	Task
Kishaann	`log_fields` copied verbatim; shown in a From-the-log box as escaped text
Cetas	Impact meter from device states and services; team progress per step; end-of-drill results by stage with missed action IDs
Cetas	Seed "who can do what" so the owner screen shows the single-point-of-failure note
Abhiram	Restyle every screen to match the wireframes; Replay mode and Simulated backup badges; QR code on the start screen
Abhiram	Kannada and Hindi on the alert card only. **A fluent speaker checks every line** before merge
Abhiram	Resilience: `usePoll` keeps the last good data through a failed request; friendly error messages; a phone that reloads mid-drill rejoins
Abhiram, Kishaann	"Practise this with the team" link on handled owner alerts selects the matching scenario
Ishaan	Wazuh lab: safe attack script making network logons with wrong passwords; confirm rule 60204 fires and the alert carries the source IP

Done when: the T1 run still passes, every screen matches its wireframe on both an Android and an iPhone, and a phone reloaded or briefly offline mid-drill recovers on its own.

Increment 3: Guarded AI rewording

Goal: the Reword toggle works with pre-written fallback wording, and the guard is in place so a model can be added later without touching the card. **When:** H12–H14, alongside Increment 4. **Gate:** T3. **Flag:** `AI_REWORD`.

Owner	Task
Kishaann	`POST /alerts/{id}/reword`: input is only the filled headline and why-text; output checks (length, numbers unchanged, no new digits or links); fallback when no provider is set, returning `source: "fallback"`
Kishaann	A provider interface with a "none" implementation; a real provider is post-demo
Abhiram	Toggle on the alert card; "Reworded" label; steps and log box never change with the toggle

Done when: with the poisoned username in the replay data, the toggle changes only the headline and why-text, and the reword code provably never receives log text.

This is the first thing to cut if the day runs late; the card works without it.

Increment 4: Approval flow, simulated

Goal: the whole approval flow works in replay mode, with the block shown as "Simulated action". **When:** H12–H16. **Gate:** T4. **Flag:** `ACTIONS`.

Building this against replay data first means the approval story is demoable even if the Wazuh lab never works. Increment 5 then only swaps the simulated block for a real one.

Owner	Task
Kishaann	Action proposer: lesson's `proposed_action`, target checks (valid IPv4, not on the never-block list), `actions` table records
Kishaann	Approve, decline, undo routes; owner-only; repeat approvals rejected
Kishaann	Background task every 10 s: applies the standing rule after 5 minutes or escalates to the backup approver; moves blocks to expired
Kishaann	Standing-rule setting with who and when; `PUT /clinic/standing-rule`
Cetas	Device statuses (OK, Under attack, Protected) driven by alerts and actions; `GET /devices`

Owner	Task
Cetas	Drill: an option may name `BLOCK_IP`; answering it never creates a real action
Abhiram	Owner live screen per the wireframe: device strip, approval card in pending, approved and declined states, countdown, "What ShieldMate did" log, standing-rule panel
Ishaan	Wazuh client interface with a fake used in replay and tests; the real one comes in Increment 5

Done when: a replayed password attack produces an approval card; Approve, Decline and the standing-rule timeout each produce the right state and log entry; and nothing is ever sent to Wazuh.

Increment 5: Live Wazuh

Goal: a real attack in the lab becomes a card on the phone, and an approved block really stops it. **When:** H14–H19, **only if** the lab gate below passes. **Gate:** T5. **Flag:** `ALERT_SOURCE=wazuh`.

Lab gate at H14: the attack script makes rule 60204 fire with a source IP, and a block sent by hand through the Wazuh API stops the script. If either fails, skip this increment and present live mode as the next step, using Increment 4's simulated block.

Owner	Task
Ishaan	Live ingest: read new alerts from the Wazuh indexer every 5 s and emit the same shape as the replay file, including the `shieldmate` counts
Ishaan	Real Wazuh client: start the block on the agent through the API (the built-in Windows script), with a 1-hour timeout; test whether undo works and report the result
Ishaan	Wazuh credentials in server config only; never sent to the browser
Kishaann	Switch the action flow from the fake client to the real one when `ALERT_SOURCE=wazuh`; replay and drill keep using the fake

Owner	Task
Abhiram	LIVE badge; Undo button shown only if undo works, otherwise "Lifts at …"
Abhiram	Backstage replay buttons stay available in live mode as the stage fallback
Cetas	Help with lab runs: time each attack-to-card and approve-to-block run and record the numbers

Done when: five lab runs in a row each show the card within 30 seconds of the attack starting and stop the attack within 30 seconds of Approve.

Increment 6: Freeze and rehearse

Goal: walk on stage with a demo that has already survived every failure you can predict. **When:** H19–H24. **Gate:** T6.

After the H19 freeze, only fix bugs that break the demo story or show on stage. No new features, no refactors, no dependency upgrades.

Hours	What	Who
H19–H21	Fix freeze-blocking bugs; full test suite on the demo laptop	All
H21	Record a clean full run as the backup video	Abhiram records, all play
H21–H23	Three timed rehearsals: normal; internet off; a phone failing mid-drill (plus the lab off, if live mode is in)	All
H23–H24	Buffer, pre-flight checklist, Reset demo, rest	All

Fix the stage roles at H19 and rehearse in them: one presenter at the laptop (also voicing Dr. Rao), two phone holders (Meena and Ravi), and Ishaan on Backstage and the lab.

Cut rules and risks

Decide cuts at the gates, by these rules, so nobody argues about them at 3 am.

If at this gate	Then
H8: Increment 1 isn't done	Everyone works on Increment 1 until it is. Drop Increment 3 entirely
H12: Increment 2 isn't done	Drop the languages and the impact meter; keep the restyle and resilience work
H14: the lab gate fails	Skip Increment 5. Ishaan moves to testing and Backstage. Live mode is pitched as the next step, shown with Increment 4's simulated block
H16: Increment 4 isn't done	Drop the standing-rule timeout; keep Approve and Decline only
H19: anything unfinished	It stays behind its flag, switched off. Nothing new merges after H19

Never cut: the role-split drill, the restore check, or Reset. They carry the story.

Risk	Effect	What to do
Wazuh lab install takes most of the night	No live mode	Start at H0; lab gate at H14 decides; Increment 4 makes live mode optional
Two VMs are too heavy for one laptop	Lab is slow or crashes	Lab runs on Ishaan's laptop only, never the presenter's
The hotspot blocks devices from reaching each other	Phones can't open ShieldMate	Use the presenter laptop's own hotspot; test at H2 with a real phone
iPhone Safari behaves differently	A screen breaks for one phone holder	Test on one Android and one iPhone at every gate
Kannada or Hindi text is wrong	Embarrassing on stage	Fluent speaker signs off, or drop it at H12
CERT-In and DPDP feedback lines are inaccurate	A judge corrects you	Check against the official texts before H19
Fatigue after H16	Bugs and bad merges late	Shift rests; freeze at H19; no merges without passing tests

After the demo: backlog

Not built in the 24 hours, in a sensible order to add next:

1. Readiness report with CERT-In draft, built from the drill's missed actions.
2. Retest history: compare runs 30 days apart to show behaviour changing.
3. Second playable drill (password guessing); the file already exists.
4. Owner PIN on approve and decline, so nobody else on the network can approve a block.
5. A real AI provider behind the existing guard.
6. More live actions: cut a PC off the network (keeping Wazuh reachable), disable an account.
7. Restore check on a real backup folder; detect whether the backup drive is mounted.
8. Websockets instead of polling, if more devices join.

Open questions

- [ ] Does the hackathon allow preparation before H0: Wazuh lab install, scenario and replay files, translations? If yes, do them before Saturday and Increment 0 shrinks to an hour.
- [ ] When exactly do H0 and the demo fall, and is the demo slot straight after H24?
- [ ] Approve and decline have no login. Is that acceptable for the demo, or is the owner PIN needed before H19?
- [ ] Who checks the Kannada and Hindi text?
- [ ] Who is the presenter?